FEASIBILITY REPORT

DataTalk: Automated Data Cleaning and Natural Language Query Engine

Author(s): Ghulam Mohyuddin
Khizar Hameed
Esha Khan
Affiliation: Department of Computer Science, National Textile University
Date: 06 September 2026

Abstract

DataTalk is an end-to-end data analysis platform that democratizes data accessibility by automating both data cleaning and natural language query execution. This feasibility report evaluates the technical, economic, operational, legal, and schedule feasibility of developing DataTalk as a Final Year Project. The system integrates two core modules: (1) CleanQuery, an automated data cleaning engine handling missing values, outliers, type inference, and format normalization, and (2) InsightFlow, a schema-aware natural language to SQL converter that eliminates hallucination through strict schema injection and validation. The project is technically feasible using Python (Flask/FastAPI backend), PostgreSQL (database), OpenAI GPT-4 API (for NLP), and Streamlit (frontend). Economically, the project requires minimal investment (0–12,000 PKR) as all core infrastructure uses open-source or free-tier SaaS. The 5–6 month development timeline is operationally feasible for a solo developer with data engineering and Python competencies. No significant legal constraints exist for an academic FYP in Pakistan. This report concludes that DataTalk is highly feasible and recommends proceeding with full development, with particular focus on robustness of the data cleaning pipeline and comprehensive error handling for non-technical users.

Keywords

Data Cleaning, Natural Language Processing, SQL Generation, Data Engineering, PostgreSQL, LLM, Schema Injection, Data Quality

1. Introduction

1.1 Background and Problem Context

According to industry research, data professionals spend approximately 80% of their working time on data cleaning and preparation before any actual analysis can begin [1]. This statistic, derived from CrowdFlower's 2016 survey of data scientists, remains consistent today. The barrier to data accessibility exists at two distinct levels: (1) data quality, raw data arrives with missing values, inconsistent formats, duplicate records, and outliers, and (2) querying complexity, extracting insights from structured databases requires SQL knowledge that most domain experts do not possess.

Current solutions address these challenges in isolation. Spreadsheet tools like Microsoft Excel (used by over 750 million users [2]) require entirely manual data cleaning. Enterprise BI platforms such as Tableau and Power BI assume data is already clean and structured. Text-to-SQL systems from major cloud providers (Amazon Athena, Snowflake, Databricks) are schema-aware but priced for large organizations at 50,000+ PKR/month [3], making them inaccessible to individual researchers, small businesses, and academic institutions. General-purpose language models can generate SQL but frequently hallucinate column names that do not exist in the actual database [4], a critical failure mode for production systems.

1.2 Problem Statement

No integrated, accessible tool currently combines automated data preprocessing with schema-aware natural language querying. The gap creates unnecessary friction:

- Non-technical users cannot clean raw data without external help
- Non-technical users cannot query databases without SQL knowledge
- Existing solutions solve only one problem in isolation
- Enterprise solutions are prohibitively expensive for individuals and SMEs
- Open-source alternatives lack robustness and error handling
- Each new dataset requires manual cleaning and re-setup

1.3 Project Objectives

- Develop an automated data cleaning pipeline handling 10+ common data quality issues
- Implement schema-aware natural language to SQL conversion with hallucination prevention
- Provide interactive visualizations for query results (bar charts, line graphs, tables)
- Create a user-friendly web interface for non-technical users
- Ensure data security with read-only query execution and destructive command blocking
- Achieve sub-3-second query response time for datasets up to 100K rows

1.4 Scope of Feasibility Study

This feasibility study examines DataTalk across five dimensions: technical feasibility (proven technology stack, architecture feasibility, complexity estimation), economic feasibility (minimal cost via free-tier SaaS and open-source tools), operational feasibility (solo developer capability, timeline realism), legal feasibility (data privacy, no regulatory barriers for academic FYP), and schedule feasibility (5–6 month development). The analysis excludes enterprise-grade deployment, HIPAA/GDPR compliance for production use, and scalability beyond 10M rows. MVP scope focuses on single-table analysis; multi-table joins are Phase 2.

2. Feasibility Analysis

2.1 Technical Feasibility

2.1.1 Recommended Technology Stack

The recommended stack prioritizes open-source libraries and free-tier SaaS for cost-effectiveness while maintaining production-grade reliability:

Component	Technology	License	Cost (PKR)
Backend API	FastAPI (Python 3.9+)	MIT License	Free
Web Frontend	React 18.x + TypeScript	MIT License	Free
Data Processing	Pandas + NumPy	BSD License	Free
Database	PostgreSQL 13+	PostgreSQL License	Free
NLP/SQL Gen	OpenAI GPT-4 API	Commercial	0-5,000 PKR*
Validation	sqlglot (SQL parser)	Apache 2.0	Free
Visualization	Plotly + D3.js	MIT License	Free
File Processing	openpyxl + pyarrow	MIT/Apache 2.0	Free
Hosting (Dev)	AWS Free Tier EC2	Free Tier	Free

Table 1: Technology Stack with Licensing and Cost Information

2.1.2 Architecture Design: Five-Stage Pipeline

DataTalk operates as a five-stage pipeline:

Stage 1: Data Ingestion: CSV, Excel, JSON, or direct database connection with file validation.

Stage 2: CleanQuery Module: Rule-based pipeline with type inference, missing value handling, outlier detection, format normalization, duplicate removal, and schema validation.

Stage 3: Database Storage & Schema Extraction: Clean data persisted to PostgreSQL with automatic schema extraction and read-only permissions.

Stage 4: InsightFlow Module (NLP to SQL): Schema injection → LLM generates SQL → sqlglot validates → keyword blacklist check → column verification → read-only execution.

Stage 5: Visualization & Response: Auto-detect result shape → select chart type → render interactive Plotly visualization → return results.

2.1.3 Core Technical Challenges and Solutions

LLM Schema Hallucination: Inject database schema into LLM system prompt as constraint. Use sqlglot parser to validate column references.

Destructive Cleaning Decisions: Never auto-delete rows. Flag issues → generate report → user reviews before committing.

Heterogeneous File Formats: Use pandas with robust parsing. Auto-detect delimiters and encodings.

SQL Validation: sqlglot library provides full SQL parsing and AST traversal (more robust than keyword blacklist).

Accuracy on Complex Queries: MVP limited to single-table SELECT queries with WHERE/GROUP BY/ORDER BY. JOINs deferred to Phase 2.

Stacked Latency: Implement result caching and progress indicators for upload (<10s), cleaning (<5s), query (<3s).

Error Communication: Every exception wrapped with user_message (plain English), system_details, and suggested_action.
Context Window Limits: Smart schema summarization for datasets >50 columns by semantic grouping.

2.1.4 Technical Risk Mitigation

Technical Risk	Probability	Mitigation Strategy
LLM API Rate Limiting	Medium	Queuing with exponential backoff. Cache results. Fallback to Ollama.
Data Privacy in API Calls	High	Never send data values to OpenAI. Only schema metadata. Comply with university policies.
Complex File Formats	Medium	Focus MVP on CSV/Excel. Graceful error handling for other formats.
Large Dataset Performance	Medium	Limit scope to <100K rows. Lazy loading. Optimize PostgreSQL with indexing.
Type Inference Errors	Low	Multiple strategies with fallback to string type. User override available.
SQL Injection	Low	Parameterized queries with psycopg2. Never concatenate user input directly.

Table 2: Technical Risk Analysis and Mitigation

Conclusion: DataTalk is TECHNICALLY FEASIBLE. Proven tech stack. Five-stage pipeline is well-defined and testable. Core challenges have clear solutions.

2.2 Economic Feasibility

2.2.1 Detailed Budget Analysis

Total project cost ranges from 0 PKR (fully open-source) to 12,000 PKR (with OpenAI API):

Category	Tool/Service	Required?	Cost (PKR)
Development	Python 3.9+ (Runtime)	No	Free
Development	FastAPI Framework	No	Free (MIT)
Development	VS Code Editor	No	Free (Open Source)
Development	Git + GitHub	No	Free (Unlimited repos)
Backend	PostgreSQL Database	No	Free (Open Source)
Backend	SQLAlchemy ORM	No	Free (MIT License)
Backend	Pandas + NumPy	No	Free (BSD License)
Backend	sqlglot SQL Parser	No	Free (Apache 2.0)
Frontend	React 18.x	No	Free (MIT License)
Frontend	Plotly Visualization	No	Free (MIT License)
NLP/LLM	OpenAI GPT-4 API	Optional	0-5,000 PKR*
NLP/LLM	Ollama (Local LLM)	Optional	Free (MIT)
Hosting	AWS Free Tier EC2	No	Free (12 months)
Hosting	Vercel (Frontend)	No	Free Tier
TOTAL MINIMUM	**(All Open Source)**		**0 PKR**

Table 3: Project Cost Breakdown (Pakistani Currency)

2.2.2 LLM Cost Analysis and Alternatives

Three LLM options exist:

OpenAI GPT-4 API: $0.15 per 1M input tokens. ~3,000 PKR for 10,000 queries. Best accuracy.

Open-Source Local Models: Ollama (Mistral, Llama 2). Zero cost. Slower (5–10s) but acceptable for MVP.

Hybrid Approach: Ollama for development. OpenAI for impressive demos.

Conclusion: DataTalk is ECONOMICALLY HIGHLY FEASIBLE. True minimum cost is 0 PKR. Optional OpenAI API adds up to 5,000 PKR. No licensing costs.

2.3 Operational Feasibility

2.3.1 Team Capacity and Required Skills

Required Competency	Current Level	Assessment
Python Programming	Advanced	Strong Match
SQL and Databases	Intermediate-Advanced	Strong Match
Data Engineering	Intermediate	Strong Match
Web API Design (REST)	Beginner	Adequate
Frontend (React)	Basic-Intermediate	Adequate
LLM Integration	Beginner-Intermediate	OpenAI docs clear
DevOps/Deployment	Beginner	AWS tutorials sufficient

Table 4: Skill Assessment for Solo Developer

2.3.2 Time Allocation and Realistic Workload

Workload: 22–25 hours/week for 5–6 months:

Month 1: Setup, Architecture Design, API Scaffolding | 18 hrs/wk
Month 2: CleanQuery Module (Data Cleaning) | 25 hrs/wk
Month 3: InsightFlow Module Part 1 (NLP to SQL) | 25 hrs/wk
Month 4: InsightFlow Module Part 2 + Validation | 25 hrs/wk
Month 5: Frontend Development + Visualization | 25 hrs/wk
Month 6: Testing, Documentation, Deployment | 22 hrs/wk

Total: ~620 hours (6 months × 23 hours/week average)

Conclusion: DataTalk is OPERATIONALLY FEASIBLE. Skills align. Workload realistic. Risks identifiable and mitigatable.

2.4 Legal and Regulatory Feasibility

2.4.1 Data Protection and Privacy

As an academic FYP, DataTalk operates under specific legal considerations:

User Data Ownership: All uploaded data remains user property. Never used for training or sharing without consent.

Storage and Retention: Data persists only during active session. Auto-delete after 30 days recommended.

Access Control: Database enforces read-only ANALYST role. No delete/modify through application.

API Call Privacy: Send only query text and schema metadata to OpenAI. Never actual data rows.

Pakistani Law Compliance: No PDPA equivalent currently in force. Academic exemptions apply.

Conclusion: DataTalk is LEGALLY FEASIBLE. No regulatory barriers. Standard data protection adequate. University approval only.

2.5 Schedule Feasibility

2.5.1 Project Phases and Timeline

Phase	Duration	Deliverables	Feasibility
Phase 0: Setup	1 week	Repo, environment, API boilerplate	Feasible
Phase 1: CleanQuery	4 weeks	5 cleaning features, unit tested	Feasible
Phase 2: InsightFlow Basics	3 weeks	SQL generation, simple queries	Feasible
Phase 3: Validation & Safety	2 weeks	Error handling, complete blacklist	Feasible
Phase 4: Frontend	3 weeks	React UI, file upload, visualization	Feasible
Phase 5: Testing & Polish	2 weeks	Unit/integration tests, documentation	Feasible
Buffer	2 weeks	Scope adjustments, technical debt	Included

Table 6: Project Phases and Feasibility

Conclusion: DataTalk is SCHEDULE FEASIBLE. 15-week critical path + 2-week buffer = 17 weeks. Fits 24-week FYP window.

3. Results and Discussion

3.1 Overall Feasibility Summary

Dimension	Rating	Evidence
Technical	HIGHLY FEASIBLE	Proven stack. Clear pipeline. Solutions identified.
Economic	HIGHLY FEASIBLE	0 PKR minimum. AWS Free Tier. No licensing.
Operational	HIGHLY FEASIBLE	Skills match. Realistic workload. Risk mitigation clear.
Legal	FEASIBLE	No barriers. Standard protection adequate.
Schedule	HIGHLY FEASIBLE	15 weeks critical path. Fits timeline.

Table 7: Overall Feasibility Assessment

3.2 Key Success Factors

1. Strong data cleaning foundation (garbage in = garbage out)
2. Robust schema injection into LLM (core technical risk)
3. Comprehensive error messages for non-technical users
4. Regular testing with real datasets
5. MVP scope discipline (defer joins, temporal queries)
6. Early supervisor feedback on design
7. Incremental deployment with user feedback

4. Conclusion and Recommendations

This feasibility study concludes that DataTalk is HIGHLY FEASIBLE as a Final Year Project in Pakistan. All five dimensions, technical, economic, operational, legal, and schedule, are feasible with clear evidence.

4.1 Final Recommendation

PROCEED WITH DEVELOPMENT OF DATATALK

DataTalk is a feasible, high-impact Final Year Project addressing a real problem with a technically sound solution. It demonstrates mastery of full-stack development, data engineering, ML integration, and user-centered design. The 5–6 month timeline is realistic. All critical success factors are within the developer's control. This project has strong potential for job applications, graduate school, and academic publication.